

让 Agent 工程有图可依

用四个主题读懂「架构图驱动的 Agentic Engineering」

整理: 项目总管 (project-overseer-001) | 日期: 2026-09-22 | 版本: v1.0

一句话: ArchGraph 以一张 ArchiMate 3.2 建模的**意图架构图**为单一事实源, 让 Agent 的设计、执行、记忆与治理都围绕同一份可执行、可追溯的图展开。

来源: archgraph.org | github.com/derekhu0002/archgraph (Apache License 2.0)

Agent 已经能写代码、能调研、能发布内容。但真正把它当作**工程**来交付时, 缺口不在「模型够不够强」, 而在四个结构性问题: **意图漂移**、**黑盒执行**、**验收靠感觉**、**经验无法沉淀**。它们的共同点是——缺少一份人和 Agent 都能读懂、且能被机器执行的「施工图」。ArchGraph 的出发点因此很朴素: **Agent 缺的不是更聪明的脑子, 而是一张施工图**。

01 项目大脑

架构知识图谱 + GraphRAG + 写入门禁 + 检索关键技术: 让项目拥有一颗可读写、可治理、可回忆的「大脑」。

02 人 + Agent 协作

以 ArchiMate 为共同心智, 在会话开始、工作中、会话结束等关键时刻提醒 Agent「记住」, 实现可控协作。

03 项目互助

Graph Store: 把「角色 / 流程 / 目标 / 约束 / 测试」的完整语义网络导出、共享、复用, 让经验跨项目流动。

04 项目集群

从个人到组织: 一套源多端运行、子图复用、组织级能力资产, 给出可落地的方法、纪律与路径。

怎么读这份文档: 四个主题各自独立成章, 每章遵循「**为什么** → **是什么** → **怎么做**」, 并配一张结构图。想快速建立整体印象, 先看每章的「一句话」与图示; 想深入, 看表格与要点。

主题一 · 项目大脑

ArchGraph 的中心是一份**意图架构图谱 (Intent Architecture Graph)**，文件形态是 `design/KG/SystemArchitecture.json`，用企业架构标准 **ArchiMate 3.2** 建模。它既是项目记忆的载体，也是检索与治理的中枢——统称「项目大脑」。

一句话：大脑由四部分构成——**架构知识图谱**（事实源）、**GraphRAG**（读写管线）、**写入门禁**（治理）、**检索关键技术**（取用）。读走「语义优先」，写走「受治理的唯一通道」。

1.1 架构知识图谱：为什么能当单一事实源

图中的每个元素都有类型（Business Actor、Business Role、Work Package、Skill、Application Component.....），元素之间是**带类型、可校验**的关系（Assignment、Triggering、Association.....），而不是自由文本。三个关键性质决定了它的地位：

性质	含义
结构化	元素与关系有类型、受关系推导矩阵约束，非法连接在 建模阶段 就被拦下，而非等到运行时。
可视化	同一张图按不同 Viewpoint (视角) 切成不同 View (视图) ；产品经理看 Application Cooperation，工程师看 Implementation and Migration，看到的是同一份数据的不同切面。
可执行	元素上挂载 可执行的验收用例 与 Skill / Rule ，提交记录回写到元素。图不只是「画出来」的，它能驱动真实工作。

1.2 GraphRAG: JSON 权威、Neo4j 从属

检索不是单纯的向量相似度，而是「**语义种子 + 语义闭包 + 结构化补全**」的组合。权威源始终是 JSON：**JSON 改了，Neo4j 跟着同步，不是反过来**。管线分五阶段：

- 嵌入向量生成**——对元素 / 关系 / 视图生成语义向量；
- 变更驱动增量索引**——不是每次全量重建；
- 种子检索**——按相似度选出语义种子；
- 目的策略闭包**——基于 ArchiMate 语义做依赖遍历，把相关子图一并拉出；
- 结构化补全**——补齐元素、关系、视图，组装成可用的上下文。

1.3 图写入门禁：治理的三道闸门

图不会腐化，靠的是「写入无害」这条纪律。所有写入都必须经过唯一通道，并依次通过三道闸门：

闸门	规则
去重 默认复用	写元素 / 关系 / 视图默认 reuse : 精确重复 (同类型 + 同名称) 直接复用, 只把既有对象挂到目标视图; 同类型语义近似 的新对象会被拦下并给出候选, 必须显式放行 (allowDuplicate + 理由) 才能创建。复用是默认, 重复要解释。
无损 写入	结构化字段按名称 合并 (testcases、视图成员、关系属性); description 等标量文本受保护, 省略不等于删除 ; 文本大幅收敛或破坏性删除必须显式确认 (acknowledgeLoss)。删减内容要么无损合并, 要么被明确承认。
墓碑 可恢复	破坏性移除 (元素 / 关系 / 视图) 被记入 NDJSON 墓碑账本 (SystemArchitecture.tombstones.ndjson), 完整对象可恢复。任何写入都不静默丢内容。

贯穿性纪律——召回优先: 一切调优不得牺牲最终检索的**检出率 / 召回率**, 只能提升; 宁可消耗性能, 也不缩小候选池、不跳过重排。这条原则同时约束读检索与写去重检索。

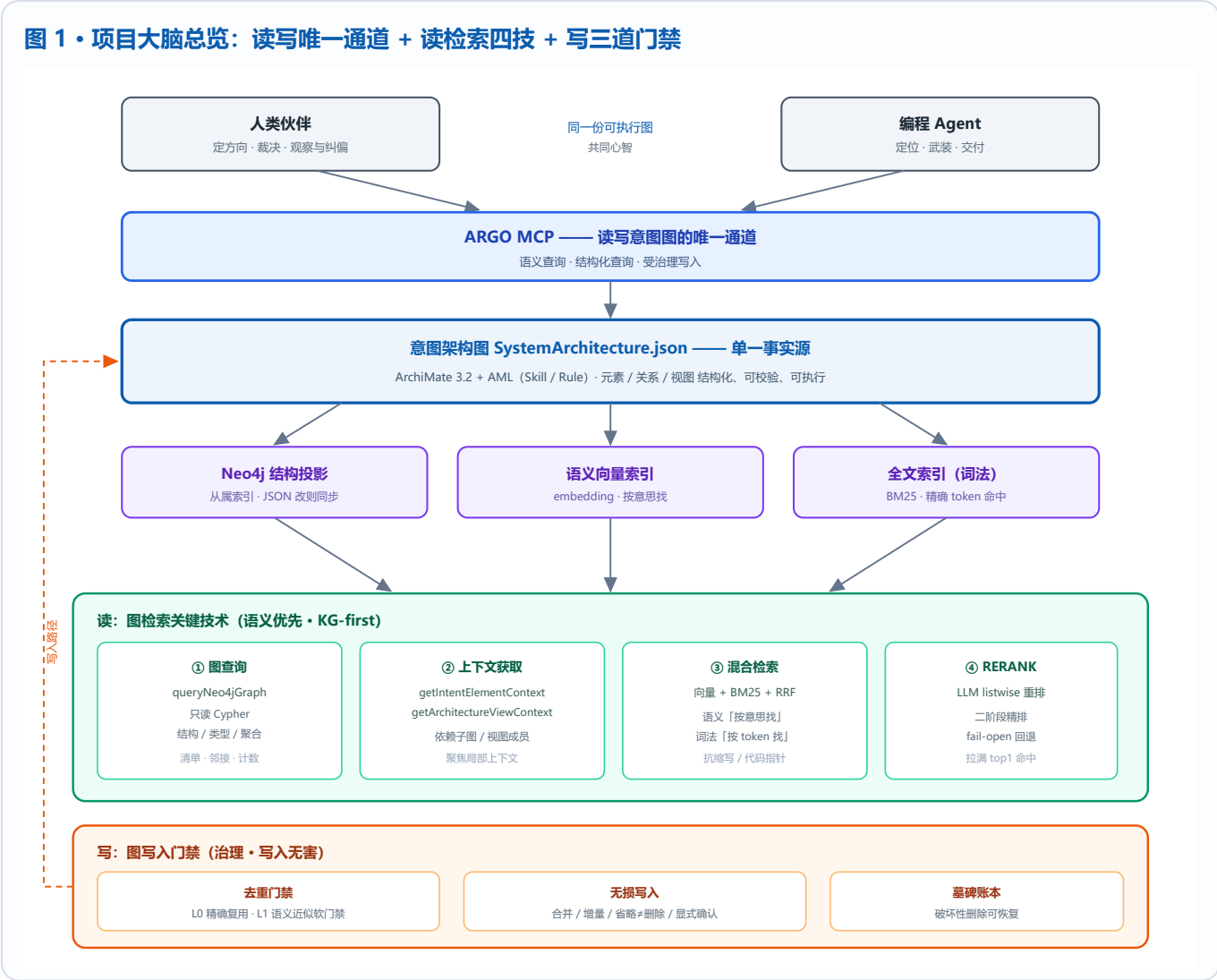
1.4 图检索关键技术：四件套

「项目大脑」要被高效取用, 靠四种互补的检索技术; 语义优先, 结构化查询作补充, 混合检索与重排负责精度。

关键技术	作用	何时用
图查询 queryNeo4jGraph	只读 Cypher, 按 结构 / 类型 做枚举、遍历、计数、聚合 (如「列出所有 Business Actor」)。只读、不写、绝不改权威图。	语义检索覆盖不到的结构化问题: 清单、关系邻接、跨类型聚合。
上下文获取 getIntentElementContext getArchitectureViewContext	以一个元素为中心展开 ArchiMate 依赖子图 (dependencyDepth / dependentDepth), 或按视图解析 完整成员 (元素 / 关系 / 父元素 / 子视图)。	已知锚点, 需要「围绕它的局部上下文」时; 两步回忆法的第二步。
混合检索 向量 + BM25 + RRF	语义向量通道「按意思找」与全文词法通道「按 token 精确找」并行, 用 RRF 融合; 对缩写、代码指针、专有名词更鲁棒。	语义命中不稳、或需要精确 token 命中时 (可开关)。
RERANK LLM listwise 重排	对候选集做一次 二阶段重排 , 把最相关的排到最前; 失败自动 fail-open 回退, 不阻塞检索。	候选集较大、需要把 top1 命中率拉满时 (默认开)。

实测口径 (仓库内评测, 非理论值): 检索基线 725 题——向量召回 **~95.3%**、混合检索 **~96.3%**; LLM 重排后 top1 命中可达 **100%**。语义种子阈值经标定为 **0.70** (无关探针 ≤ 0.69 、相关目标 ≥ 0.83 之间取最大召回); 相关性最终由 Agent 自判——只有「零相关」才拒答, 低分命中不轻易丢弃。

图 1 • 项目大脑总览：读写唯一通道 + 读检索四技 + 写三道门禁



读路径（自下而上取用）与写路径（受治理的唯一通道）共同构成项目大脑；去重、无损、墓碑三道闸门保证图不腐化。

主题二 · 人 + Agent 协作

协作的难点不是「Agent 能不能干」，而是「人和 Agent 是否在同一页上」。ArchGraph 用两件事解决：一套**共同心智**（同一张 ArchiMate 图），以及一套**记忆节拍**（在关键时刻提醒 Agent 记住）。

一句话：人看全局、Agent 看局部，但看的是**同一份数据**；目标对齐不靠开会，而是**结构性的**。会话开始恢复记忆、工作中即时记账、会话结束幕等收口——让 Agent 永远不会「忘记上次做到哪儿」。

2.1 共同心智：ArchiMate 作为人与 Agent 的公共语言

ArchGraph 没有另造 DSL，而是把 ArchiMate 3.2 扩展为 Agent 建模语言（AML），只新增 Skill 与 Rule 两个标准里确实没有的元素。这带来三重「共同心智」：

维度	人	Agent
看什么	Viewpoint 分层的全局视图，俯瞰职责与协作	任务相关的局部子图（视图成员 / 依赖闭包）
怎么用	观察 · 约束 · 纠偏（改 description / testcases / Assignment）	定位元素 → 自武装 Skill/Rule → 测试先行 → 提交回登记
共同点	面对同一张图、同一套元素与关系；产品侧（Application Component）与 Agent 侧（Business Actor / Role / Skill / Rule）通过 Work Package 相连，成为「目标 ↔ 执行者」的枢纽。	

2.2 记忆节拍：在关键时刻提醒 Agent 记住

人的记忆分「意识区 / 需回忆 / 笔记档案」，Agent 亦然。ArchGraph 用**三层记忆**承载，并在三个关键时刻触发写入或加载：

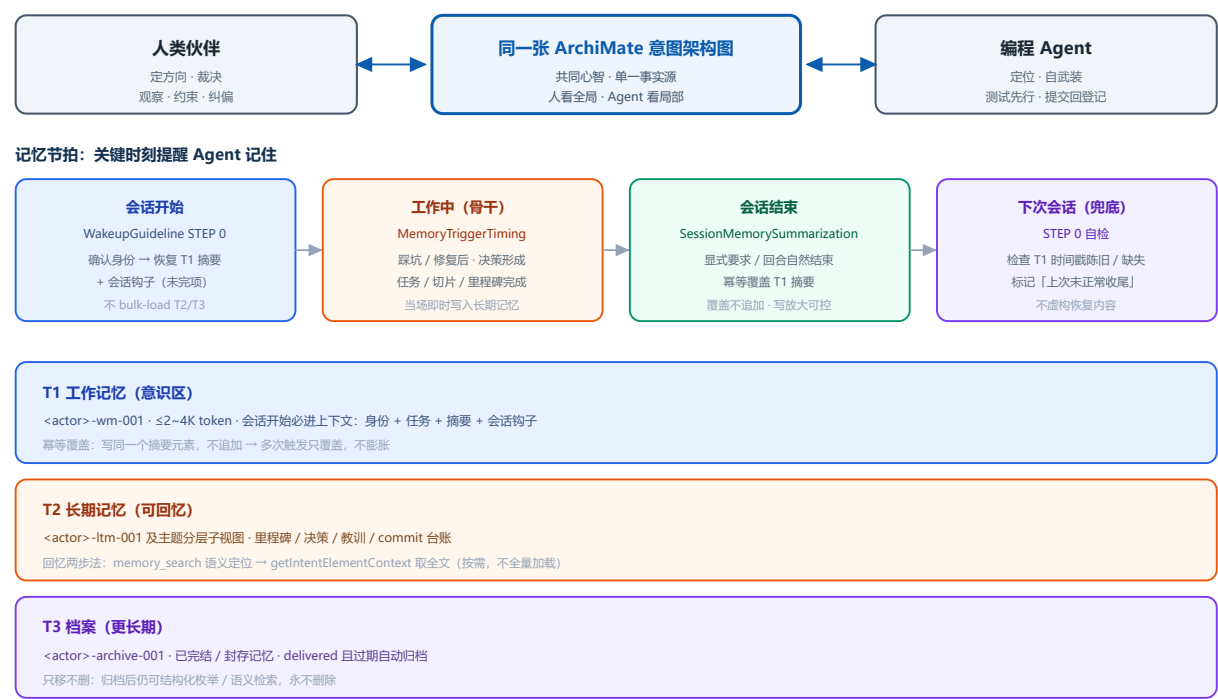
时刻	机制	做什么
会话开始	WakeupGuideline · STEP 0	先确认「我是哪个 Business Actor」，再 只加载 T1 工作记忆摘要 + 会话钩子 （上次未完成项 / 关键决策指针）；不 bulk-load T2/T3，避免撑爆上下文。
工作中	MemoryTriggerTiming	踩坑 / 修复后、「关键决策」形成时、任务 / 切片 / 里程碑完成时—— 当场立即写入 长期记忆。这是最可靠的骨干：即使会话摘要漏写，关键内容早已入账。
会话结束	SessionMemorySummarization	人类显式要求收尾，或回合自然结束时， 幕等覆盖 同一个 T1 摘要元素（覆盖而非追加，写放大可控）。不苛求精确检测「会话结束」——关键内容已由上一行兜底。
下次会话	STEP 0 自检（兜底）	检查 T1 摘要时间戳是否陈旧 / 缺失，标记「上次未正常收尾」并提示人类；上下文已丢也不虚构恢复。

层级	视图约定	内容与访问方式
T1 工作记忆 意识区	<actor>-wm-001	身份 + 职责、当前任务、最近记忆摘要、会话钩子；≤2~4K token，会话开始必进上下文。
T2 长期记忆 可回忆	<actor>-ltm-001（及主题分层子视图）	全部里程碑 / 决策 / 教训 / commit 台账；命中才加载。 回忆两步法 ：memory_search 语义定位 → getIntentElementContext 取全文。
T3 档案 更长期	<actor>-archive-001	已完结 / 封存的旧记忆；delivered 且过期自动归档。 只移不删 ，显式检索才读。

2.3 掌控力：可观察、可约束、可纠偏

- **CoreRules 闭环**：找元素 → 走 MCP → 测试先行 → 提交登记 → 记忆回写 → 持续合规。每一步都有约束。
- **委派隔离 (CoperationGuideline)**：Agent 不能越权，必须通过正式委派协作；Developer 不能干 Reviewer 的活，除非有明确的委派关系。每个 Actor 有独立的会话与记忆，互不干扰。
- **人的介入点**：**观察** (Viewpoint 俯瞰全局)、**约束** (改 Role description / testcases / Skill)、**纠偏** (改 Assignment 重新分配任务)。

图 2 · 共同心智与记忆节拍：人与 Agent 围绕同一张图协作



共同心智让目标对齐结构化；记忆节拍（开始 / 工作中 / 结束 / 自检）保证 Agent 不丢上下文、不失忆。

主题三 · 项目互助

每个项目都在重复发明 Agent 的工作方法：规则、流程、技能，散落各处、缺乏共享。ArchGraph 用 **Graph Store（图资产仓库 / 子图库）** 让**架构决策的结构化表达**跨项目流动——这就是「项目互助」。

一句话：共享的最小单位不是一段 prompt，而是一个**子图工作包**——含角色、流程、目标、约束、测试的完整语义网络。它可导出、可复用、可组合，让一个项目的经验成为另一个项目的起点。

3.1 可分享子图工作包是什么

通过仓库内的导出能力，从 design/KG/SystemArchitecture.json 中递归抽取一个自治的子图，包含该工作所需的全部语义要素：

- **角色：**Business Actor / Role 及其 Assignment；
- **流程：**Process / Function 与 Triggering / Flow 关系；
- **目标与约束：**Goal / Requirement / Constraint / Rule；
- **能力与验证：**Skill 与 GIVEN-WHEN-THEN 验收用例。

导出时保留 schema_id、parent 层级与 subdiagram_views，因此导入到另一个项目后仍是**结构化、可导航、可执行的**，而不是一堆需要人肉重新理解的文字。

3.2 共享闭环：导出 → 治理 → 发布 → 复用

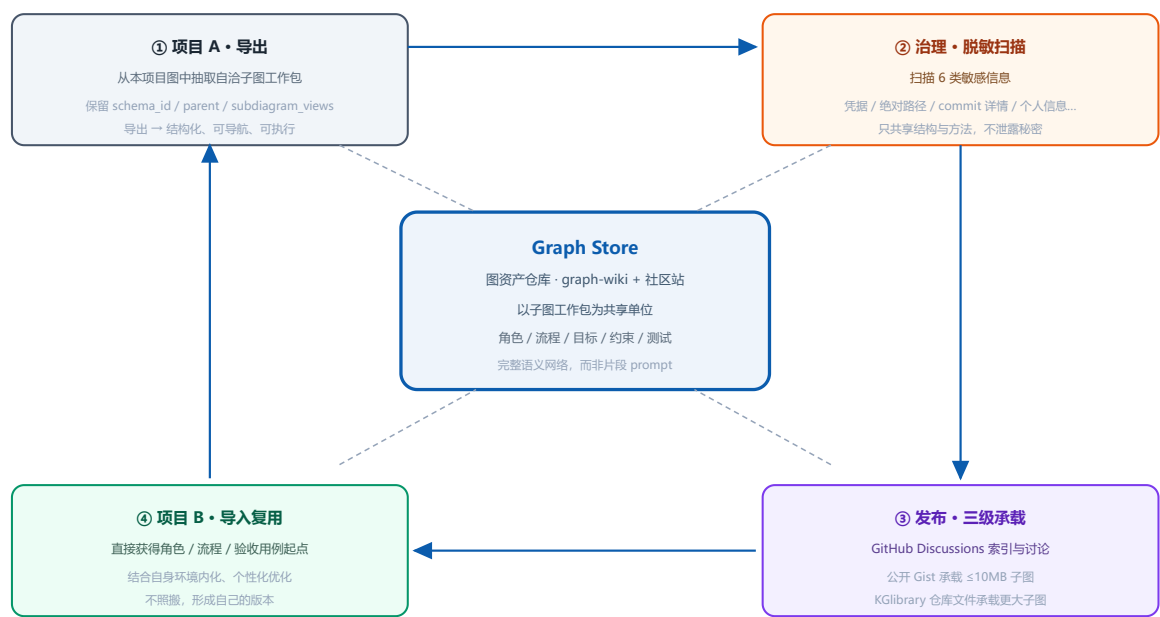
环节	说明
导出	从本项目图中选出一个认为可分享子图工作包（角色 / 流程 / 目标 / 约束 / 测试）。
治理（脱敏）	发布前扫描 6 类敏感信息 （凭据、绝对路径、commit 详情、个人信息等），确保只共享 结构与方法 ，不泄露秘密。
发布（三级承载）	① GitHub Discussions 索引与讨论；② 公开 Gist 承载 ≤10MB 子图；③ KGlibrary / 仓库文件 承载更大的子图。
复用与内化	另一个项目导入子图，直接获得角色、流程与验收用例的起点；并结合自身环境 内化、个性化优化 ，而非照搬。

边界与取舍：社区以 GitHub Discussions 为载体（零成本、零运维、生态内），代价是对单一平台存在依赖；子图默认以公开 Gist 承载，超过其约 10MB 单文件上限则改用仓库文件。这些取舍是**明示的**，而非隐藏风险。

3.3 与业界共享机制对比

方案	共享的是什么	局限
Hugging Face	模型参数	黑盒：知道「能做什么」，不知道「怎么做、为什么这么做」。
LangChain / AutoGen	编排逻辑（调用链）	白盒但仅限调用链，不含角色职责、目标、约束与测试的完整语义。
ArchGraph	架构决策的结构化表达：角色 / 流程 / 目标 / 约束 / 测试的完整语义网络	需要 ArchiMate 心智门槛；以平台（GitHub）为社区载体。

图 3 • Graph Store 共享闭环：子图跨项目流动



共享闭环：项目 A 导出 → 治理脱敏 → 三级发布 → 项目 B 导入内化。中心始终是「子图工作包」这一结构化语义网络。

主题四 · 项目集群

一个人用 ArchGraph 是一个项目；一个组织用 ArchGraph，是一群项目共享同一套语言、纪律与能力资产。本章给出从**个人提效**到**组织提效**的方法、纪律与路径。

一句话：ArchGraph 的杠杆在于**可复用**——一套源多端运行、子图跨项目复用、能力资产沉淀到组织。个人提效靠「一张图 + 一个 Agent」，组织提效靠「一族已治理的可复用资产」。

4.1 路径：四个成熟度阶段

阶段	谁在用	关键动作	收益
L1 起步器	个人 · 单项目	安装 ARGO 工具链；argo init 建立意图图；1 人 + 1 Agent 围绕一张图工作。	Agent 有据可依、可追溯；单项目不再「跑偏无感、验收靠感觉」。
L2 多角色	个人 · 单项目多 Agent	用 Business Actor / Role 建模多个角色，通过正式委派协作；每个 Actor 独立记忆。	复杂任务可拆分、可隔离；角色能力（Skill）可组合。
L3 项目互助	团队 · 多项目	把子图工作包导出到 Graph Store，跨项目复用；对照自身环境内化。	不再重复发明工作方法；一个项目的经验成为另一个项目的起点。
L4 组织集群	组织 · 项目群	统一语言与纪律；组织级 Skill / Rule / 验收用例资产库 + 治理规范；一份源多 Harness 运行。	组织级能力资产沉淀；规模化交付质量可回归、可审计、可沉淀。

4.2 方法：三根杠杆

杠杆一 · 一套源多端运行

一条 argo-deploy 把 ARGO MCP、Skills、Rules、Agents 部署到 GitHub Copilot、Cursor、OpenCode、DeepSeek Harness、OpenClaw。一份源文件，多端运行——团队不再为「每个 Harness 各维护一套」付出重复成本。

杠杆二 · 子图复用

以角色 / 流程 / 目标 / 约束 / 测试为单位的**子图工作包**，可导出、可组合、可再导出。组织把「最佳实践」沉淀为可挂载的资产，新项目直接站在它之上。

杠杆三 · 统一语言与枢纽元素

ArchiMate 3.2 + AML 让产品侧与 Agent 侧同图；**Work Package** 作为枢纽，同时指向产品组件与 Agent 技能，形成 **Task** → **Skill** → **Capability** 的语义链。技能不再是散落的 prompt 片段，而是可追溯、可复用的组织能力。

4.3 纪律：规模化不腐化的四道约束

纪律	规模化时的作用
定位先行	动手前先在图里找到（或创建）对应元素与视图；元素必须属于至少一个视图，不存在「图之外的孤儿元素」——多项目下依然边界清晰。
验收先行	每个元素挂载 GIVEN-WHEN-THEN、 可执行的 验收用例；改动前识别受影响用例、改动后跑回归。多个项目共享用例时，质量可横向对齐。
变更可证	每次提交把 commit id + 文件路径 回登记到对应元素，元素拥有自己的变更时间线——谁、何时、改了什么，一目了然。
写入无害	去重（复用优先）+ 无损（省略≠删除）+ 墓碑（破坏性删除可恢复）。多人 / 多 Agent 并发写图时，图不会因重复或误删而腐化。

叠加两条协作纪律：**记忆分层**（T1/T2/T3，避免上下文膨胀与知识丢失）与**委派隔离**（各 Actor 独立会话与记忆，不越权、不互相干扰）。



四阶段递进：个人单项目 → 单项目多角色 → 团队多项目（项目互助） → 组织项目群；四道工程约束贯穿始终。

4.4 组织提效的落点：能力资产化

- **Skill / Rule 是一等公民**：它们物化为文件系统上的具体文件（如 SKILL.md、*.instructions.md），并可被元素沿 Association 关联挂载。Agent 领取 Work Package 后**自武装**，主动获取所需能力。
- **验收用例是资产**：每个元素的验收用例是**代码**而非文档，可跨项目复用与回归，是组织质量的一致性来源。
- **记忆是资产**：踩坑经验、关键决策写入长期记忆，随项目沉淀并可跨会话召回；子图导出时可携带经验，形成「带坑的技能」。
- **可审计**：变更可证 + 写入无害，使组织级并发协作仍然可追溯、可恢复、可审计。

结语 · 快速开始

ArchGraph 用企业架构的严谨方法论 (ArchiMate + TOGAF) 来工程化 Agent 开发, 而不是把 Agent 当作「Prompt 工程」的玩具。它直指当前 Agent 生态的三大痛点: **不可见** (行为黑盒) → 架构图让其可见; **不可控** (自由发挥难纠偏) → 验收 + 追溯让人类保持掌控; **不可复用** (Skills 散落) → 图谱化让知识资产可组合、可共享。

```
npm install -g archgraph-argo
argo-deploy
```

安装后打开项目、启动编程 Agent, 它会自动: **先定位** (在图里找到任务背后的架构元素) → **再武装** (获取该元素关联的 Skill 与 Rule) → **测试先行** (用 GIVEN-WHEN-THEN 驱动实现) → **可追溯** (提交并回登记到图谱)。

语义 (Graph RAG) 检索需额外配置 Neo4j 与向量引擎; Embedding 可指向任意 **OpenAI 兼容端点**, 便于离线 / 内网 / 私有化部署。其余能力开箱即用。

资源	地址
官网	https://archgraph.org
代码仓库	https://github.com/derekhu0002/archgraph
社区站	https://argo.derekworkspacev5.com/archgraph/ (文档 / 社区 / 子图库 / 博客)
图资产仓库	https://github.com/derekhu0002/graph-wiki

ArchGraph 以 **Apache License 2.0** 开源。欢迎试用、提 Issue——一起把 Agent 工程, 从「玄学」变成「工程」。